Correcciones para R2 — notebook v1

Campo	Valor
Revisado	`src/jose/practica-002-v1/` (notebook, README, datos y resultados)
Alcance	Secciones 4 a 6
Fecha	11 de noviembre de 2026
Veredicto	Lo técnico está impecable; queda **una decisión del equipo por aplicar** y un ajuste de nombre

La v1 ejecuta de punta a punta (22 celdas, counts 1–22, 0 errores) y las secciones 1–3 llegan **byte-idénticas** a la v4 aprobada de R1. Verifiqué además: los 17 `.joblib` cargan con sus hiperparámetros correctos (`svr_mejor.joblib` = rbf, C=10, `epsilon` =0.1, `gamma` =0.01), `resultados_modelos.csv` tiene 17 filas ordenadas por RMSE con las columnas del plan, los artefactos de EDA regenerados son idénticos a los promovidos, `datos.sha256` pasa y `requirements.txt` coincide con el raíz. La selección del `champion` por CV y no por test (6.2–6.3) es exactamente lo que pide el plan. Solo dos puntos.

P0

1. El registry no se limpia en re-ejecuciones

Por qué. La celda 5.1 borra el modelo registrado y todos los runs antes del barrido ("se borran el modelo registrado y los runs de una ejecución anterior") para que una re-ejecución termine siempre en v1–v17. El equipo decidió lo contrario: el registry **es el histórico de entrenamientos y no se limpia** — re-ejecutar agrega versiones nuevas y el alias `champion` pasa a la versión vigente, que es justamente el propósito del alias (§7.7 del plan: "no hay que tocar código cuando cambie el mejor modelo"). Con el borrado, cualquier re-ejecución (tuya, la de R3 al agregar la sección 7, o la del profesor al reproducir) destruye el histórico que la guía pide conservar. La decisión ya está escrita en el plan (§7.6) y en `docs/entorno.md`.

Qué.

- 5.1: eliminar el bloque de borrado (el `if cliente.search_registered_models(...)` con `delete_registered_model` y el `for run in ...: cliente.delete_run(...)`). El run de prueba se queda: es la validación de la tarea 1 del plan y solo se borra a sí mismo.
- Capturar la numeración relativa al inicio de 5.1:

```
version_inicial = len(cliente.search_model_versions(f"name='{NOMBRE}'"))
```

- 5.2: el `assert` pasa a relativo:

```
assert len(barrido) == 16 and list(barrido.index) == list(range(version_inicial + 1,
version_inicial + 17))
```

- 5.3: el baseline deja de ser "la versión 1" fija:

```
punto_partida = pd.DataFrame([ingenuo, barrido.loc[barrido.index[0], list(ingenuo)]],
                             index=["ingenuo (promedio_movil_7d)", f"v{barrido.index[0]}:
linear, C=1"]).round(2)
```

- 6.2: `assert version == 17 ...` pasa a relativo; el `set_registered_model_alias` no cambia (mover el alias a la nueva versión es lo esperado):

```
assert version == version_inicial + 17 and resumen_versiones["rmse_cv"].idxmin() == version
```

- 6.4: filtra las versiones a las de esta ejecución (la tabla queda con 17 filas aunque el registry acumule) y relativiza los criterios:

```
mios = set(resumen_versiones["run"])
registradas = [v for v in cliente.search_model_versions(f"name='{NOMBRE}'") if v.run_id in
mios]
```

```
criterios = {
    "los 17 runs de esta ejecución terminaron y están activos": all(
        cliente.get_run(r).info.status == "FINISHED" for r in mios),
    "17 versiones nuevas y consecutivas": list(tabla_versiones.index) ==
list(range(version_inicial + 1, version_inicial + 18)),
    "todas las versiones con tags kernel y C": tabla_versiones[["kernel",
"C"]].notna().all().all(),
    "todos los runs con rmse, mae, r2, mape y rmse_cv": tabla_versiones[["rmse", "mae", "r2",
"mape", "rmse_cv"]].notna().all().all(),
    "champion = la versión del gridsearch de esta ejecución":
str(cliente.get_model_version_by_alias(NOMBRE, "champion").version) == str(version),
    ...
}
```

- Markdown 5.1, 6.4 y "Entrega a R3 y R4": reemplazar "la celda borra el registro y los runs de una ejecución anterior" y "volver a ejecutar la sección 5 borra y recrea el registro: siempre quedan 17 runs activos y las versiones 1 a 17" por: re-ejecutar agrega 17 versiones nuevas y mueve `champion` a la nueva versión del `GridSearchCV`; el histórico queda visible en `mlflow ui`.
- Para la v2, corre **una sola vez con un `mlflow.db` limpio** (bórralo a mano antes de ejecutar; tu db local ya quedó con el estado de la v1). Así el notebook entregado muestra v1–v17; las re-ejecuciones posteriores ya acumulan sin romper nada.

P1

2. El notebook se entrega como `src/svr_ventas.ipynb`

Por qué. El plan (§7.2) y las entregas aprobadas de R1 (v2–v4) usan un único nombre y ubicación: `src/svr_ventas.ipynb`. La promoción a la raíz copia ese archivo sin renombrar, R3 lo recibe con el nombre documentado y el notebook no cambia de nombre entre versiones. (La v1 de R1 también llegó con otro nombre, `r1-v1.ipynb`, y se normalizó desde la v2.)

Qué. Mover `modelado_mlflow.ipynb` a `src/svr_ventas.ipynb` dentro de `src/jose/practica-002-v1/` (misma estructura que las entregas de R1). No hace falta re-ejecutar por esto: `raiz_proyecto()` resuelve la misma raíz desde `src/` que desde la carpeta de la entrega. Actualizar las referencias al nombre en el README (bloque Estado, árbol del repo y tabla Estado por rol).

Cierre

- □ 5.1 sin bloque de borrado; `version_inicial` capturada antes del barrido.
- □ Asserts y criterios de 5.2, 5.3, 6.2 y 6.4 relativos; tabla filtrada a esta ejecución.
- □ Markdown 5.1, 6.4 y entrega a R3/R4 sin "borra y recrea el registro".

□ Notebook en `src/svr_ventas.ipynb` ; README sin `modelado_mlflow.ipynb` .

□ Ejecutado una vez con `mlflow.db` limpio: 0 errores, HTML muestra v1–v17 y `champion` en la versión del `GridSearchCV` ; `sha256sum -c` sigue pasando.